Nested Queries

SQL

SQL tutorial gives unique learning on Structured Query Language and it helps to make practice on SQL commands which provides immediate results. SQL is a language of database, it includes database creation, deletion, fetching rows and modifying rows etc. SQL is an ANSI (American National Standards Institute) standard, but there are many different versions of the SQL language.

Why SQL?

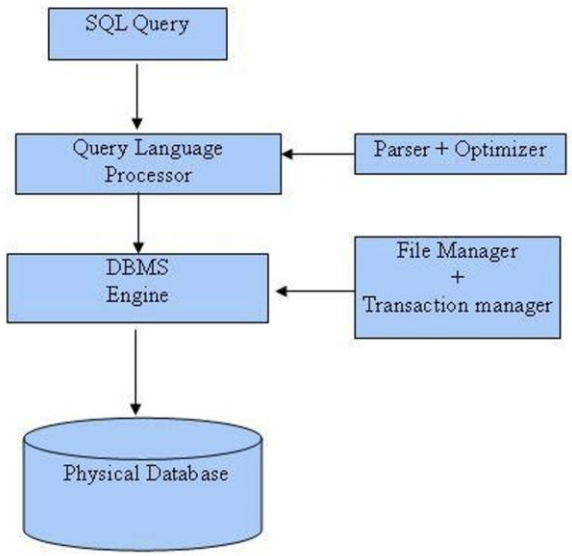
- Allows users to access data in relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in the database and manipulate that data.
- Allows embedding within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create views, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views

SQL Process

When you are executing an SQL command for any RDBMS, the system determines the best way to carry out your request and SQL engine figures out how to interpret the task.

There are various components included in the process. These components are Query Dispatcher, Optimization Engines, Classic Query Engine and SQL Query Engine, etc. Classic query engine handles all non-SQL queries, but SQL query engine won't handle logical files.

Following is a simple diagram showing SQL Architecture:



SQL Commands

In previous manual we covered DDL & DML, in this manual we'll cover DQL

DQL - Data Query Language:

Command	Description
SELECT	Retrieves certain records from one or more tables

Details of DQL

---

1. Multi-Step Queries with Subqueries

SQL subqueries enable complex queries by performing multiple steps, such as filtering data based on aggregated values or computing derived results.

1.1 Subquery in WHERE Clause

A subquery in the `WHERE` clause allows filtering results based on another query.

Example **Scenario:**
Find students who have paid the highest tuition fee.

```
SELECT student_name, tuition_fee
FROM Payments
WHERE tuition_fee = (SELECT MAX(tuition_fee) FROM Payments);
```

Explanation:

- The inner query `(SELECT MAX(tuition_fee) FROM Payments)` fetches the highest tuition fee.
- The outer query retrieves students who have paid this fee.

1.2 Subquery in FROM Clause

A subquery in the `FROM` clause allows treating its result as a temporary table.

Example **Scenario:**
Find the average tuition fee of students who have paid more than $10,000.

```
SELECT AVG(high_payers.tuition_fee) AS avg_high_payers
FROM (SELECT tuition_fee FROM Payments WHERE tuition_fee > 10000) AS high_payers;
```

Explanation:

- The subquery `(SELECT tuition_fee FROM Payments WHERE tuition_fee > 10000)` filters students who paid more than $10,000.
- The outer query calculates their average tuition fee.

1.3 Subquery in SELECT Clause

A subquery in the `SELECT` clause allows computing derived values.

Example **Scenario:**
List all students along with the highest tuition fee paid.

```
SELECT student_name, tuition_fee,
    (SELECT MAX(tuition_fee) FROM Payments) AS highest_fee
FROM Payments;
```

Explanation:

- The subquery `(SELECT MAX(tuition_fee) FROM Payments)` calculates the highest tuition fee.
- The outer query lists all students along with this value.

---

2. Correlated Subqueries

A correlated subquery references columns from the outer query and executes once per row.

2.1 Finding Students Who Paid Above the Average Fee

```
SELECT student_name, course_name, tuition_fee  FROM Enrollments e1

WHERE tuition_fee = (SELECT MAX(tuition_fee) FROM Enrollments e2 WHERE e1.student_id = e2.student_id);
```

Explanation:

- The subquery finds the highest tuition fee per student.
- The outer query retrieves the corresponding course.

---

3. Advanced MSSQL Functions with Subqueries

3.1 Using ROW_NUMBER() to Rank Students by Tuition Fee

```
SELECT student_name, tuition_fee,

    ROW_NUMBER() OVER (ORDER BY tuition_fee DESC) AS rank

FROM Payments;
```

Explanation:

- `ROW_NUMBER()` assigns a unique rank to each student based on the tuition fee.
- Students with the highest fees are ranked first.

3.2 Using PARTITION BY to Calculate Running Totals

```
SELECT student_name, tuition_fee,

    SUM(tuition_fee) OVER (PARTITION BY course_id ORDER BY student_name) AS running_total

FROM Payments;
```

Explanation:

- `SUM() OVER (PARTITION BY course_id ORDER BY student_name)` calculates a running total of tuition fees per course.
- Each row shows the cumulative sum of fees paid by students in a course.

3.3 Using CTEs with Recursive Queries

Example **Scenario:**
Find a hierarchical list of course prerequisites.

```
WITH CourseHierarchy AS (

  SELECT course_id, prerequisite_id

  FROM CoursePrerequisites

  WHERE prerequisite_id IS NULL
```

```
    UNION ALL
    SELECT c.course_id, ch.prerequisite_id
    FROM CoursePrerequisites c
    INNER JOIN CourseHierarchy ch ON c.prerequisite_id = ch.course_id
)
SELECT * FROM CourseHierarchy;
```

Explanation:

- A Common Table Expression (CTE) recursively retrieves course prerequisites.
- The `UNION ALL` statement enables traversing hierarchical relationships.

3.4 Using LEAD() and LAG() for Comparative Analysis

```
SELECT student_name, tuition_fee,
    LAG(tuition_fee) OVER (ORDER BY tuition_fee DESC) AS previous_fee,
    LEAD(tuition_fee) OVER (ORDER BY tuition_fee DESC) AS next_fee
FROM Payments;
```

Explanation:

- `LAG()` fetches the previous row's tuition fee.
- `LEAD()` fetches the next row's tuition fee.
- This allows comparisons between consecutive rows.

3.5 Using DENSE_RANK() for Ranking Without Gaps

```
SELECT student_name, tuition_fee,
    DENSE_RANK() OVER (ORDER BY tuition_fee DESC) AS rank
FROM Payments;
```

Explanation:

- **`DENSE_RANK()` ranks students by tuition fee, ensuring consecutive ranking numbers without gaps.**

3.6 Using NTILE() for Percentile Distribution

```
SELECT student_name, tuition_fee,

    NTILE(4) OVER (ORDER BY tuition_fee DESC) AS quartile
FROM Payments;
```

Explanation:

- `NTILE(4)` divides students into four quartiles based on tuition fee distribution.